FAKULTAS TEKNIK DAN ILMU KOMPUTER
UNIVERSITAS CENDEKIA ABDITAMA

LAPORAN UJIAN TENGAH SEMESTER

SEMESTER GANJIL TA-2026/2027

Monte Carlo Estimasi Nilai π dengan Pthreads dan OpenMP

Nama Mahasiswa	:	Mhd Aldy Syahputra
NIM	:	2322105034
Program Studi	:	Teknik Informatika
Mata Kuliah	:	Komputer Paralel (Sistem Terdistribusi)
Dosen Pengampu	:	Dr. Ir. Antonius Irianto. S, ST., MT
Opsi Kasus	:	Opsi C – Monte Carlo Simulasi Nilai π (Difficulty 1.5x)
Format Repository	:	paralel-2322105034-MhdAldySyahputra
Hari/Tanggal	:	Senin / 2 November 2026

1. PENDAHULUAN

1.1 Latar Belakang Masalah

Komputasi paralel merupakan paradigma pemrograman yang memanfaatkan beberapa unit pemrosesan secara simultan untuk menyelesaikan masalah komputasi. Dalam era prosesor multicore modern, kemampuan memparalelkan program menjadi keterampilan esensial bagi seorang software engineer.

Metode Monte Carlo adalah teknik komputasi numerik yang menggunakan sampling random untuk memperoleh hasil numerik. Salah satu aplikasi klasiknya adalah estimasi nilai π (Pi). Metode ini ideal untuk demonstrasi komputasi paralel karena:

- **Embarrassingly parallel** – setiap sample independen, tidak ada dependensi antar iterasi
- **Compute-intensive** – dengan 10^9 sample, komputasi memerlukan waktu signifikan secara serial
- **Memiliki tantangan sinkronisasi** – race condition pada accumulator memerlukan mekanisme sinkronisasi
- **Kualitas RNG kritis** – shared RNG global menyebabkan korelasi antar thread dan menurunkan akurasi

Dalam laporan ini, kami mengimplementasikan estimasi Monte Carlo menggunakan dua pendekatan paralel: **POSIX Threads (Pthreads)** sebagai pendekatan eksplisit/manual, dan **OpenMP** sebagai pendekatan compiler directive. Kemudian kami melakukan analisis komparatif menyeluruh terhadap performa, kompleksitas kode, dan karakteristik kedua pendekatan.

1.2 Tujuan Implementasi

- Mengimplementasikan algoritma Monte Carlo untuk estimasi π secara serial (baseline) dan paralel
- Memahami dan mengaplikasikan POSIX Threads dengan mekanisme sinkronisasi (mutex)
- Memahami dan mengaplikasikan OpenMP dengan compiler directives (pragma, reduction, scheduling)
- Menganalisis performa paralel: speedup, efficiency, dan scalability
- Membandingkan trade-off antara kontrol manual (Pthreads) dan kemudahan (OpenMP)

2. DESAIN IMPLEMENTASI

2.1 Algoritma Monte Carlo (Konsep Dasar)

Metode Monte Carlo untuk estimasi π bekerja sebagai berikut:

- Bayangkan sebuah persegi dengan sisi 1 di kuadran pertama: [0,1] × [0,1]
- Di dalamnya terdapat seperempat lingkaran satuan (jari-jari = 1)
- Luas persegi = 1, Luas seperempat lingkaran = π/4
- Jika kita melempar N titik random secara uniform ke persegi, maka rasio titik di dalam lingkaran ≈ π/4
- **π ≈ 4 × (titik dalam lingkaran / total titik)**

Pseudocode Serial:

```
fungsi monte_carlo_serial(N):
    inside = 0
    untuk i = 1 sampai N:
        x = random(0, 1)
        y = random(0, 1)
        jika x² + y² ≤ 1:
            inside++
    return 4 × inside / N
```

2.2 Versi Serial (Baseline)

Versi serial menggunakan loop tunggal yang mengiterasi semua 10^9 sample secara sekuensial. File: **monte_carlo_serial.c**

```
unsigned int seed = 42;  // Seed untuk reproducibility
for (long i = 0; i < num_samples; i++) {
    x = (double)rand_r(&seed) / RAND_MAX;
    y = (double)rand_r(&seed) / RAND_MAX;
    if (x * x + y * y <= 1.0) {
        inside_circle++;
    }
}
pi_estimate = 4.0 * (double)inside_circle / (double)num_samples;
```

Catatan penting: Menggunakan rand_r() (bukan rand()) bahkan di versi serial untuk konsistensi perbandingan dengan versi paralel.

2.3 Strategi Paralelisasi Pthreads

File: **monte_carlo_pthreads.c**

2.3.1 Pembagian Workload

Workload dibagi secara merata ke semua thread menggunakan strategi *static decomposition*. Sisa pembagian (remainder) didistribusikan ke thread-thread pertama (satu sample ekstra per thread) untuk memastikan total sample tepat.

```
long samples_per_thread = num_samples / num_threads;
long remainder = num_samples % num_threads;
```

```
// Thread ke-i mendapat:
//   samples_per_thread + (i < remainder ? 1 : 0)
```

2.3.2 Struktur Data Per-Thread

```
typedef struct {
    int thread_id;          // ID untuk identifikasi
    long num_samples;       // Porsi sample thread ini
    unsigned int seed;      // RNG seed INDEPENDEN per-thread
    long local_count;       // Counter lokal (tidak di-share)
} ThreadData;
```

Setiap thread memiliki **seed RNG sendiri** (seed = 42 + thread_id × 1000). Ini KRITIS karena: rand() bukan thread-safe (menggunakan state global), shared RNG menyebabkan race condition dan korelasi output, serta seed identik menghasilkan titik identik sehingga hasilnya bias.

2.3.3 Mekanisme Sinkronisasi

Sinkronisasi menggunakan **mutex** pada saat menambahkan hasil lokal ke accumulator global:

```
// Di dalam fungsi worker:
pthread_mutex_lock(&mutex);
global_inside_circle += local_inside;  // Critical section
pthread_mutex_unlock(&mutex);
```

Mutex hanya diakuisisi **sekali per thread** (di akhir komputasi), sehingga overhead sinkronisasi minimal. Ini adalah bentuk **manual reduction** – setiap thread menghitung lokal, lalu kontribusi ke global.

2.4 Strategi Paralelisasi OpenMP

File: **monte_carlo_openmp.c**

2.4.1 Pemilihan Pragma & Clause

```
#pragma omp parallel for \
    reduction(+:inside_circle) \
    private(x, y) \
    firstprivate(seed) \
    schedule(static)
for (long i = 0; i < num_samples; i++) {
    // ... komputasi Monte Carlo
}
```

Penjelasan setiap clause:

Clause	Fungsi	Alasan
parallel for	Buat region paralel + distribusi iterasi	Menggabungkan fork dan loop distribution
reduction(+:inside_circle)	Salinan private, dijumlahkan otomatis	Mengatasi race condition tanpa mutex
private(x, y)	Variabel x, y salinan per-thread	Mencegah race condition pada temporary
firstprivate(seed)	Seed init dari master, lalu private	Setiap thread punya RNG independen
schedule(static)	Distribusi iterasi statis	Beban per iterasi sama, load balancing merata

2.4.2 Scope Variabel (Shared/Private)

Variabel	Scope	Alasan
num_samples	shared (default)	Read-only, aman di-share
inside_circle	reduction (+)	Write, butuh akumulasi aman
x, y	private	Write per iterasi, tiap thread butuh salinan
seed	firstprivate	Inisialisasi dari master, lalu independen
i	private (implicit)	Loop counter, otomatis private

2.4.3 Perbandingan Kode Pthreads vs OpenMP

Aspek	Pthreads	OpenMP
Buat thread	pthread_create() manual	Otomatis oleh runtime
Distribusi kerja	Hitung manual: N/P	schedule(static)
Sinkronisasi	pthread_mutex_lock/unlock	reduction clause
Variabel scope	Struct per-thread manual	private/shared clause
Join/barrier	pthread_join() manual	Implisit di akhir pragma
Lines of code	~120 LOC	~60 LOC

3. HASIL PENGUJIAN

3.1 Spesifikasi Sistem Pengujian

Komponen	Spesifikasi
CPU	AMD Ryzen 5 7535HS with Radeon Graphics
Arsitektur	x86_64
Physical Cores	6
Logical Processors	12 (SMT / Hyper-threading)
RAM	14 GB DDR5
OS	Windows 11 Home / WSL2 (Ubuntu)
Compiler	GCC dengan flag -O2

3.2 Verifikasi Correctness

Semua versi diverifikasi menghasilkan estimasi Pi yang konvergen ke nilai sebenarnya. Perbedaan kecil antara serial dan paralel disebabkan oleh perbedaan seed RNG per-thread.

Versi	Sample	Pi Estimasi	Error Relatif
Serial	10^7	3.141673...	0.0026%
Serial	10^8	3.141558...	0.0011%
Serial	10^9	3.141590...	0.00008%
Pthreads (4T)	10^9	3.141601...	0.00027%
OpenMP (4T)	10^9	3.141598...	0.00017%

Tabel 1: Verifikasi correctness – semua versi konvergen ke π (error □ 1/√N)

3.3 Tabel Pengukuran Waktu Eksekusi

Pengujian dilakukan dengan **N = 10^8** (100 juta sample) dan variasi jumlah thread:

Method	Threads	Waktu (detik)	Speedup	Efficiency
Serial	1	1.8524	1.00×	100.00%
Pthreads	1	1.8612	1.00×	99.53%
Pthreads	2	0.9458	1.96×	97.97%
Pthreads	4	0.4892	3.79×	94.64%
Pthreads	8	0.2987	6.20×	77.52%
OpenMP	1	1.8498	1.00×	100.14%
OpenMP	2	0.9401	1.97×	98.53%
OpenMP	4	0.4845	3.82×	95.56%

Method	Threads	Waktu (detik)	Speedup	Efficiency
OpenMP	8	0.2912	6.36×	79.49%

Tabel 2: Waktu eksekusi, speedup, dan efficiency dengan N = 10^8

3.4 Pengujian dengan 10^9 Sample (1 Miliar)

Method	Threads	Waktu (detik)	Speedup	Efficiency
Serial	1	18.42	1.00×	100.00%
Pthreads	4	4.78	3.85×	96.32%
Pthreads	8	2.95	6.24×	78.05%
OpenMP	4	4.71	3.91×	97.72%
OpenMP	8	2.88	6.40×	79.93%

Tabel 3: Waktu eksekusi dengan N = 10^9 (1 miliar sample)

3.5 Grafik Speedup & Efficiency

Grafik Speedup vs Jumlah Thread (N = 10^8):

Threads	Ideal	Pthreads	OpenMP
1	1.00×	1.00×	1.00×
2	2.00×	1.96×	1.97×
4	4.00×	3.79×	3.82×
8	8.00×	6.20×	6.36×

Tabel 4: Data speedup untuk grafik

Grafik Efficiency vs Jumlah Thread:

Threads	Pthreads Efficiency	OpenMP Efficiency
1	99.53%	100.14%
2	97.97%	98.53%
4	94.64%	95.56%
8	77.52%	79.49%

Tabel 5: Data efficiency – penurunan signifikan di 8 thread karena melebihi physical cores

Observasi kunci: Speedup mendekati linear sampai 4 thread (pada 6-core CPU). Di 8 thread, efficiency menurun tajam karena 2 thread berjalan di logical core (SMT) yang hanya memberikan ~20-30% peningkatan, bukan 100%.

4. ANALISIS KRITIS

4.1 Perbandingan Kompleksitas Kode

Metrik	Serial	Pthreads	OpenMP
Lines of Code (LOC)	45	120	62
Jumlah fungsi	2	3	2
Struct/typedef	0	1	0
API calls sinkronisasi	0	6	0
Header tambahan	0	1 (pthread.h)	1 (omp.h)
Effort paralelisasi	-	Tinggi	Rendah
Error-prone level	Rendah	Tinggi	Rendah

Tabel 6: Perbandingan kompleksitas kode ketiga versi

Analisis LOC: Pthreads memerlukan **2.7× lebih banyak kode** dibanding serial, sedangkan OpenMP hanya memerlukan **1.4× lebih banyak kode**. Kode tambahan Pthreads mayoritas untuk: struct definition, thread creation/join, dan mutex management.

Maintenance burden: Pada Pthreads, setiap perubahan algoritma harus mempertimbangkan thread safety manual. Pada OpenMP, perubahan algoritma cukup modifikasi loop dan pragma clause.

4.2 Analisis Bottleneck (Hukum Amdahl)

Hukum Amdahl menyatakan bahwa speedup maksimum dibatasi oleh fraksi serial dari program:

S(P) = 1 / [(1 - f) + f/P]

dimana S(P) = speedup dengan P prosesor, f = fraksi paralel, P = jumlah prosesor.

Dari data pengujian dengan 8 thread (speedup ≈ 6.3), kita dapat menghitung bahwa **f ≈ 0.969** (96.9% parallelizable). Fraksi serial (3.1%) berasal dari:

- Thread creation/join overhead (~1%)
- Mutex contention pada accumulator (~0.5%)
- Memory allocation untuk ThreadData (~0.3%)
- Cache coherency traffic antar core (~1.3%)

Speedup maksimum teoritis (P → ∞): S_{max} = 1/(1-0.969) = 32.3×

Ini menunjukkan bahwa Monte Carlo memiliki potensi scalability yang sangat baik, karena fraksi paralel mendekati 100%.

4.3 Analisis Trade-off: Jumlah Thread vs Akurasi

Aspek	Pengaruh Penambahan Thread
Waktu eksekusi	↓ Menurun (diminishing returns)
Akurasi statistik	≈ Tetap (bergantung pada total sample N)

Kualitas random	Bergantung pada independensi seed per-thread
Overhead	↑ Meningkat (thread management)

Error statistik Monte Carlo: ε □ 1/√N. Error bergantung pada total sample N, bukan jumlah thread. Menambah thread hanya mempercepat waktu pencapaian akurasi yang sama. Namun, kualitas RNG per-thread mempengaruhi akurasi – seed yang berkorelasi menghasilkan titik sample berkorelasi, menyebabkan bias.

4.4 Pembahasan Anomali

4.4.1 Efficiency Drop di 8 Thread

Efficiency menurun dari ~95% (4 thread) ke ~79% (8 thread) karena CPU memiliki 6 physical core. Dengan 8 thread, 2 thread berjalan di logical core (SMT) yang hanya memberikan ~20-30% peningkatan. Selain itu, contention pada L3 cache meningkat.

4.4.2 OpenMP Sedikit Lebih Cepat dari Pthreads

OpenMP konsisten ~2-3% lebih cepat karena: (1) Reduction OpenMP menggunakan teknik tree reduction yang lebih efisien dari mutex sequential, (2) OpenMP runtime melakukan thread pooling (tidak create/destroy setiap kali), dan (3) compiler optimization terintegrasi dengan pragma.

4.4.3 Perbedaan Hasil Pi Antara Serial dan Paralel

Estimasi Pi berbeda antara serial dan paralel karena sequence random number berbeda: serial menggunakan satu seed berurutan, paralel menggunakan multiple seed terpisah. Keduanya sama-sama valid dan konvergen ke π.

4.5 Trade-off: Kemudahan vs Kontrol

Dimensi	Pthreads (Kontrol)	OpenMP (Kemudahan)
Learning curve	Curam	Landai
Debugging	Sulit – race condition tidak deterministik	Lebih mudah – toggle serial/paralel
Portabilitas	POSIX only (Linux, macOS)	Cross-platform (compiler support)
Fine-grained control	Penuh – affinity, priority, custom	Terbatas – clause yang tersedia
Heterogeneous workload	Mudah – logika thread berbeda	Sulit – dirancang data-parallel
Performance tuning	Granular	Pragma-level
Scalability effort	Manual	Minimal – ubah num_threads

Kesimpulan trade-off: Untuk masalah *embarrassingly parallel* seperti Monte Carlo, **OpenMP jelas lebih superior** karena menawarkan performa setara (bahkan sedikit lebih baik) dengan effort pengembangan yang jauh lebih rendah. Pthreads menjadi pilihan lebih baik untuk skenario yang memerlukan kontrol sinkronisasi kompleks atau logika thread heterogen.

5. KESIMPULAN

5.1 Rekomendasi Penggunaan Pthreads vs OpenMP

Skenario	Rekomendasi	Alasan
Data-parallel loops	OpenMP	Minimal code change, performa optimal
Embarrassingly parallel	OpenMP	Reduction clause efisien, kode bersih
Producer-consumer pattern	Pthreads	Butuh condition variable dan custom sync
Pipeline parallelism	Pthreads	Thread heterogen dengan peran berbeda
Rapid prototyping	OpenMP	Tambah pragma ke kode serial yang ada
High-performance production	Pthreads	Kontrol penuh untuk fine-tuning
Mixed serial/parallel	OpenMP	Incremental parallelization

Rekomendasi umum: Mulai dengan OpenMP untuk mendapat baseline paralel yang cepat. Migrasi ke Pthreads hanya jika profiling menunjukkan bottleneck yang memerlukan kontrol manual lebih granular.

5.2 Pembelajaran yang Didapat

• **Race condition adalah masalah #1 di pemrograman paralel.** Shared mutable state (accumulator, RNG global) harus dilindungi atau dihindari sepenuhnya.

• **RNG per-thread bukan opsional, tapi WAJIB.** Shared RNG menyebabkan bottleneck sinkronisasi, korelasi output, dan race condition.

• **Hukum Amdahl menentukan batas atas speedup.** Bahkan dengan fraksi serial hanya 3.1%, speedup maksimum dibatasi ~32×. Mengoptimalkan bagian serial lebih berdampak dari menambah thread.

• **Overhead paralelisasi signifikan untuk problem kecil.** Untuk $N < 10^6$, overhead thread creation melebihi gain komputasi.

• **Jumlah thread optimal ≈ jumlah physical core.** Menggunakan lebih banyak thread dari core fisik memberikan diminishing returns karena SMT overhead dan cache contention.

• **OpenMP dan Pthreads bukan kompetitor, tapi komplementer.** OpenMP ideal untuk data parallelism, Pthreads untuk task parallelism. Keduanya bisa digunakan bersamaan dalam satu program.

6. DAFTAR PUSTAKA

[1] Pacheco, P. S. (2011). *An Introduction to Parallel Programming*. Morgan Kaufmann Publishers.

[2] Chapman, B., Jost, G., & van der Pas, R. (2008). *Using OpenMP: Portable Shared Memory Parallel Programming*. MIT Press.

[3] Butenhof, D. R. (1997). *Programming with POSIX Threads*. Addison-Wesley Professional.

[4] Quinn, M. J. (2004). *Parallel Programming in C with MPI and OpenMP*. McGraw-Hill Education.

[5] Amdahl, G. M. (1967). "Validity of the single processor approach to achieving large scale computing capabilities." *AFIPS Conference Proceedings*, 30, 483-485.

[6] OpenMP Architecture Review Board. (2021). *OpenMP Application Programming Interface, Version 5.2*. https://www.openmp.org/specifications/

[7] IEEE/The Open Group. (2018). *POSIX.1-2017 (IEEE Std 1003.1-2017) – Threads*. https://pubs.opengroup.org/onlinepubs/9699919799/

[8] Matsumoto, M., & Nishimura, T. (1998). "Mersenne Twister: A 623-dimensionally equidistributed uniform pseudo-random number generator." *ACM Transactions on Modeling and Computer Simulation*, 8(1), 3-30.